Carta Digital PNK Security — Manual de implementación (3ª entrega)

Carlos Araya · Matías Puelles — Desarrollo Seguro, INACAP La Serena

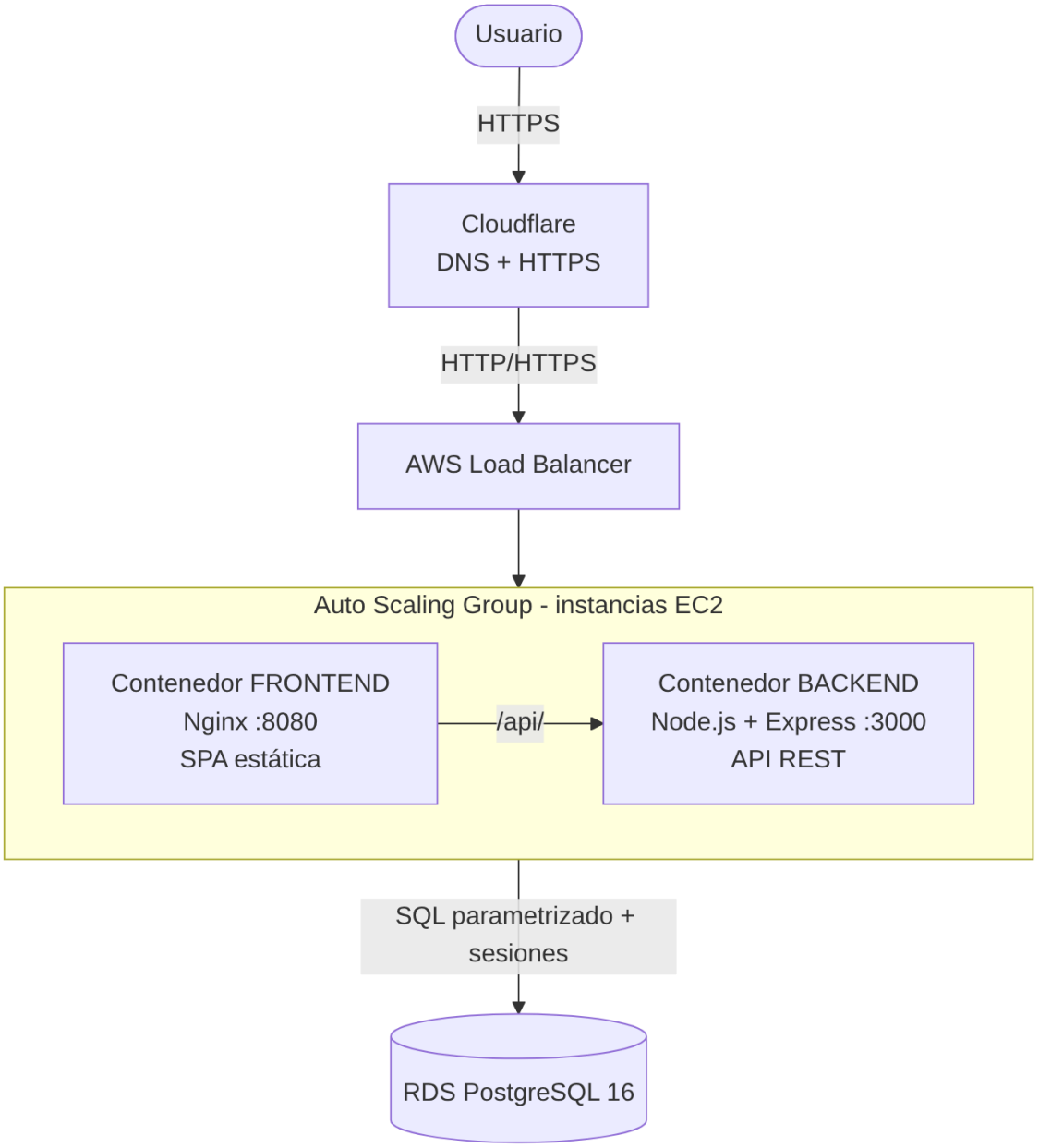
1. Objetivo y alcance

Esta entrega reconstruye la carta digital como **Front-End + Back-End separados**, con **Node.js + Express + PostgreSQL**, patrón **MVC**, **Docker** (un contenedor para el frontend y otro para el backend), **tres ambientes** (desarrollo, QA, producción), publicación con **Git** y despliegue en **AWS** (RDS PostgreSQL, Auto Scaling, Load Balancer) detrás de **Cloudflare**. El paso de Cloudflare es manual; todo lo demás es código.

2. Stack

Capa	Tecnología
Frontend	HTML + CSS + JavaScript (módulos ES, sin compilar) servido por Nginx
Backend	Node.js 22 + Express 4 (API REST JSON)
Base de datos	PostgreSQL 16 (RDS en AWS)
Contenedores	Docker, docker compose (dev, qa, prod)
Infraestructura	CloudFormation (`infra/aws/carta-digital.yml`) y `infra/deploy.sh`

3. Arquitectura



Arquitectura

MVC. *Modelos*: `backend/src/models` (SQL parametrizado). *Controladores*: `backend/src/controllers` + `routes/api.js`. *Vista*: la SPA de `frontend/public` (HTML/CSS/JS) que consume la API.

Sesiones en PostgreSQL (`connect-pg-simple`): así varias instancias detrás del Load Balancer comparten el inicio de sesión.

3.1 API REST

Método y ruta	Descripción	Auth
`GET /api/live`	Liveness (lo usa el Load Balancer; no consulta la BD)	no
`GET /api/health`	Estado y conexión a la BD	no
`GET /api/sesion`	Usuario actual y token CSRF	no

Método y ruta	Descripción	Auth
`POST /api/login` · `POST /api/logout`	Iniciar / cerrar sesión	CSRF
`GET /api/restaurantes` · `/:id`	Listado y detalle	no
`GET /api/restaurantes/:id/carta`	Cartas, categorías, productos y destacados	no
`GET /api/restaurantes/:id/items?ids=1,2`	Precios vigentes de productos (para el pedido)	no
`GET /api/restaurantes/:id/comentarios`	Comentarios	no
`POST /api/restaurantes/:id/comentarios`	Publicar comentario	sesión + CSRF

3.2 Tres ambientes

Ambiente	Archivo	Puerto	Notas
Desarrollo	`docker/docker-compose.dev.yml`	8080	Código montado, backend con `--watch`, datos y usuario demo
QA	`docker/docker-compose.qa.yml`	8081	Imágenes como producción; secretos propios
Producción	`docker/docker-compose.prod.yml`	80	Solo lectura, sin privilegios, sin datos demo; BD = RDS

4. Procedimiento (solo local, sin AWS)

Requisitos: Docker con Compose v2 y Git.

```
git clone https://github.com/damianpavon/entrega-numero-3.git
cd entrega-numero-3

# Desarrollo
cp docker/env/dev.env.example docker/env/dev.env
docker compose --env-file docker/env/dev.env -f docker/docker-compose.dev.yml up --build
# http://localhost:8080   usuario demo: demo@carta.cl / Demo#Dev2026

# QA
cp docker/env/qa.env.example docker/env/qa.env      # cambiar DB_PASSWORD y SESSION_SECRET
docker compose --env-file docker/env/qa.env -f docker/docker-compose.qa.yml up -d --build
# http://localhost:8081
```

Genera secretos con `openssl rand -hex 32`. Para crear un usuario en QA/producción: `docker compose ... exec backend node scripts/crear-usuario.js "Nombre" correo@dominio.cl "Clave larga"`.

Pruebas automáticas (Node 22, sin dependencias): `cd backend && npm test` y `cd frontend && node --test test/*.test.js`.

5. Despliegue en AWS

El script **`infra/deploy.sh`** crea los contenedores, publica las imágenes en ECR, crea la base de datos y toda la infraestructura (CloudFormation). Requiere AWS CLI con credenciales, Docker y una región.

```
export AWS_REGION=us-east-1
./infra/deploy.sh qa   --aws                      # QA
CERTIFICATE_ARN=arn:aws:acm:... ./infra/deploy.sh prod --aws   # Producción (exige HTTPS)
./infra/deploy.sh qa --aws --destroy              # eliminar el stack
```

Qué crea el stack: RDS PostgreSQL 16 cifrado (clave en Secrets Manager), Application Load Balancer, Auto Scaling Group (1–3 instancias, escala por CPU 60 %) con plantilla que arranca los dos contenedores, roles IAM mínimos, acceso por SSM (sin SSH) y grupos de seguridad encadenados: Internet → ALB → Nginx (8080) → backend (3000, solo red interna) → RDS (5432).

Rollback: `IMAGE_TAG=<etiqueta anterior> ./infra/deploy.sh prod --aws`.

5.1 Cloudflare (paso manual)

1. Cloudflare → dominio → **DNS** → agregar **CNAME** `carta` apuntando al `AlbDnsName` que imprime el script, con proxy activado.
2. **SSL/TLS** → modo **Full (strict)** si se desplegó con `CERTIFICATE_ARN`; si no, *Flexible* (solo QA).
3. **Edge Certificates** → activar *Always Use HTTPS*.
4. Verificar `https://carta.tudominio.cl/api/health` → `{"status":"ok"}`.

6. Variables de entorno

Variable	Descripción
`NODE_ENV`	`development` o `production`
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`	Conexión a PostgreSQL (`DB_SSL=true` en RDS)
`SESSION_SECRET`	Obligatorio (≥ 32 caracteres) fuera de desarrollo
`COOKIE_SECURE`	`true` si se llega por HTTPS
`TRUST_PROXY`	Proxies delante del backend: 1 local, 3 en AWS (Cloudflare + ALB + Nginx). Es parámetro `TrustProxy` del stack
`RUN_MIGRATIONS`	Aplica el esquema al iniciar (con bloqueo, seguro con varias instancias)
`SEED_DEMO`	Carga restaurantes de ejemplo; el usuario demo solo existe si se define `DEMO_USER_PASSWORD`

7. Seguridad

Riesgo	Control aplicado
Inyección SQL	Consultas parametrizadas en todos los modelos

Riesgo	Control aplicado
XSS	El frontend construye el DOM con `textContent`; CSP estricta (`script-src 'self'`, sin estilos ni scripts inline)
CSRF	Token por sesión en cabecera `X-CSRF-Token`; cookie `SameSite=Lax`
Contraseñas	bcrypt; comparación con hash falso si el usuario no existe (mismo tiempo de respuesta)
Fijación de sesión	Sesión nueva al iniciar sesión
Fuerza bruta	Límite de 10 intentos / 15 min en login y 300 / min en la API
Manipulación de precios	El carrito guarda solo ids; los precios se consultan al servidor
Suplantación en comentarios	El autor sale de la sesión, no del formulario
Secretos	Nunca en Git (`.gitignore`); en AWS: Secrets Manager; sin `set -x` en el arranque
Contenedores	Usuario sin privilegios, sistema de archivos de solo lectura, `cap_drop ALL`, `no-new-privileges`
Red	Backend y BD no son públicos; RDS cifrada

8. Verificaciones realizadas y límites

Realizado en el entorno de desarrollo del trabajo:

- 19 pruebas del backend y 9 del frontend (todas pasan).
- Las consultas SQL reales de los modelos se ejecutaron contra PostgreSQL 16 (incluido un intento de inyección guardado como texto).
- Prueba en navegador (Chromium) de la SPA contra los controladores y modelos reales: listado, carta, pedido, precios del servidor, login, comentario con HTML guardado como texto, cierre de sesión, rutas inexistentes y ausencia de violaciones de CSP.
- `docker compose config` de los tres ambientes, validación de la plantilla CloudFormation y `bash -n` del script.

No se pudo ejecutar (el entorno no tenía acceso a Docker en ejecución, a `npm install` ni a AWS): el arranque real con Express instalado, `docker build`, `nginx -t` y el despliegue en AWS. Se recomienda ejecutar primero el ambiente de desarrollo local y QA antes de AWS; si `npm ci` crea un `package-lock.json`, súbalo al repositorio.

Recomendación para producción: limitar el Security Group del ALB a los rangos IP de Cloudflare (https://www.cloudflare.com/ips/), porque con `TRUST_PROXY=3` un acceso directo al ALB podría falsificar `X-Forwarded-For` y esquivar el límite de intentos. La app se conecta a RDS con el usuario administrador generado por Secrets Manager; un usuario de aplicación con privilegios mínimos queda como mejora.

9. Problemas frecuentes

Síntoma	Causa y solución
`SESSION_SECRET debe definirse…`	Falta el secreto en el `.env`
No se mantiene la sesión por HTTP local	`COOKIE_SECURE=true` sin HTTPS: ponerlo en `false`
502 en `/api/`	El backend no está sano: `docker compose logs backend`
El script rechaza el `.env`	Aún contiene valores `CAMBIAR`

10. Siguiente entrega (referencia)

La 4ª entrega agrega Jenkins con complementos OWASP y MITRE; el repositorio ya separa pruebas, imágenes y plantilla de infraestructura para integrarlas en un *pipeline*.